

Programação Orientada a Objetos**Lista de Exercícios 03****Aula 06 – Herança**

- 1) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
 - a) Construtores das superclasses não são herdadas por subclasses
 - b) Um relacionamento parte-de é implementado via herança
 - c) Uma classe Carro tem um relacionamento é-um com as classes Volante e Freios
 - d) Quando uma subclasse redefine um método da superclasse usando a mesma assinatura, ela é dita por sobrecarregar o método da superclasse.
- 2) Discuta de que maneira a herança promove a reutilização de software, economiza tempo durante o desenvolvimento de um programa e ajuda a evitar erros.
- 3) Discuta como é feita a chamada de construtores em subclasses. O código abaixo irá funcionar? Justifique.

```
public class TesteConstrutorPaiFalho{
    public TesteConstrutorPaiFalho(String nomePai){
        System.out.println("Construtor pai falho: "+ nomePai);
    }
}

public class TesteConstrutorFilhoFalho extends TesteConstrutorPaiFalho {
    public TesteConstrutorFilhoFalho (){
        System.out.println("Construtor filho falho");
    }

    public static void main(String args[]){
        TesteConstrutorFilhoFalho t = new TesteConstrutorFilhoFalho();
    }
}
```

- 4) Escreva uma hierarquia de herança para as classes Quadrilateral, Trapezoid, Parallelogram, Rectangle e Square. Use Quadrilateral como a superclasse da hierarquia. Crie e use uma classe Point para representar os pontos em cada forma. Faça a hierarquia o mais profunda possível, isto é, com muito níveis. Especifique as variáveis de instância e os métodos para cada classe. As variáveis de instância private de Quadrilateral devem ser os pares de coordenadas x-y para os quatro pontos que delimitam o Quadrilateral. Escreva um programa que instancia objetos de suas classes e gera saída da área de cada objeto (exceto Quadrilateral).



- 5) Elabore um aplicativo de simulação de controle de contas bancárias. Todas as contas devem ter o nome do cliente, número da conta, saldo e métodos para sacar e depositar o dinheiro. Além disso, especialize a classe para que possa representar diferentes tipos de contas, neste caso, conta poupança e conta especial. Conta poupança possui um dia de rendimento e método de calcular novo saldo, dado o rendimento (%). Saques não podem deixar poupança negativa. Cada conta especial possui um limite que pode ser sacado depois do saldo estar zerado. Crie um aplicativo para testar a classe com um menu que receba o nome do usuário. Caso usuário já exista, abra outro menu que o permita sacar, depositar ou verificar dia de rendimento (caso conta seja do tipo poupança). Caso usuário não exista, abra opção de criar um dos dois tipos de conta. Caso o usuário entrada seja "admin", liste todos os correntistas com seus respectivos tipos de conta.

Aula 07 – Polimorfismo, Classes Abstratas, Interfaces e Exceções

- 6) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- Todos os métodos em uma classe abstrata devem ser declarados como métodos abstratos
 - Invocar um método exclusivo da subclasse por uma variável do tipo da subclasse não é permitido
 - Se uma superclasse declara um método abstrato, uma subclasse deve implementar aquele método
 - Um objeto de uma classe que implementa uma interface pode ser considerado como um objeto daquele tipo da interface.
- 7) Quais tipos de exceção podem ser capturadas utilizando o bloco abaixo? Por que não é adequado utilizar este tipo de tratamento de exceções?

```
try{
    } catch(Exception e){}
```

- 8) Uma companhia paga seus empregados em uma base semanal. Os empregados são de quatro tipos. Empregados salarizados são pagos um salário semanal independentemente do horário de horas trabalhadas. Empregados por hora são pagos pela quantidade de horas trabalhadas e recebem hora extra (1,5x o salário de uma hora tradicional) para cada hora depois de 40 horas trabalhadas. Empregados de comissão são pagos por uma porcentagem de suas vendas e um empregado de comissão salariado a porcentagem de suas vendas mais um salário base. Escreva um método que receba qualquer um destes objetos e calcule seu pagamento semanal. (Não sobrecarregue os métodos! Implemente somente um método que trate esses objetos de forma polimórfica.)
- 9) Altere o exercício 8 (ganhos semanais para os empregados) e crie uma classe de exceção `IllegalWorkingConditions`. No método de calcular os ganhos de cada tipo de empregado, avalie se os empregados de comissão recebem menos que 10% de suas vendas e se os empregados por hora trabalharam mais de 60 horas na semana. Caso alguma dessas situações sejam encontradas, lance a exceção que você acabou de criar e a trate adequadamente. Insira uma `String` na exceção, explicando a exceção que ocorreu e a exiba para o usuário no bloco `catch`. Para avisar que os ganhos de algum empregado já foram calculados, crie um bloco `finally` junto ao `try-catch` que, ao terminar o cálculo de ganhos, seja normalmente ou depois de avisar ao usuário de que o empregado está em situação de trabalho irregular, avise que os seus ganhos já estão calculados.

Nota: Ao se criar um aplicativo Java, é importante não permitir que o programa pare de executar, a não ser que o próprio usuário solicite isto (o mesmo deverá ser válido para o projeto). Portanto, neste exercício, caso alguma exceção seja lançada para algum empregado, avise o usuário com uma mensagem, não calcule os ganhos deste empregado, mas continue calculando dos outros em situações regulares de trabalho.